
Project Summary

MiniLMS AI is an intelligent Learning Management System built with ASP.NET Core MVC that enhances traditional course management with artificial intelligence. It solves the problem of static learning by integrating Google Gemini AI and Qdrant Vector Database to provide semantic search, document summarization, and context-aware question answering. Designed for educators and students, the platform offers a scalable solution for managing educational content while enabling advanced, AI-driven interaction with course materials.

Code Overview

This repository contains **MiniLMS AI**, an intelligent Learning Management System built with ASP.NET Core MVC. It integrates Google Gemini AI and Qdrant Vector Database to provide features like document summarization and semantic search.

Core & Infrastructure

- **Program.cs**: The application entry point that configures the HTTP pipeline, registers services for Dependency Injection (Repositories, Services, AI), and sets up Identity with role-based authorization policies.
- **MiniLms.csproj**: Defines the project dependencies, including Semantic Kernel for AI integration, Entity Framework Core for SQL Server, and PdfPig for document processing.
- **appsettings.json**: Contains configuration settings for the SQL Server connection string and Google Gemini AI model parameters.

Controllers (Presentation Layer)

- **AccountController.cs**: Manages user authentication, including registration, login, and logout using ASP.NET Identity.
- **CourseController.cs**: Handles administrative actions for courses, such as creating, editing, and viewing course details and associated documents.
- **StudentController.cs**: Provides CRUD operations for managing student profiles.
- **EnrollmentController.cs**: Manages the relationship between students and courses, allowing for new enrollments and tracking.
- **HomeController.cs**: Serves the main dashboard and landing pages of the application.

Services (Business Logic)

- **AIService.cs**: Interfaces with Google Gemini to generate embeddings for text, create automated quizzes, and summarize educational content.
- **CourseService.cs**: Implements business logic for course management, utilizing Eager Loading to fetch related lessons and documents.
- **VectorDbService.cs & VectorSyncService.cs**: (Inferred) Manage the synchronization of course content into the Qdrant vector database for semantic search capabilities.
- **LessonService.cs & StudentService.cs**: Handle the specific business rules and data orchestration for lessons and student entities respectively.

Data & Repositories (Data Access)

- **ApplicationDbContext.cs:** The EF Core database context that defines the schema and relationships for Users, Courses, Lessons, Enrollments, and Documents.
- **GenericRepository.cs:** Implements a base repository pattern to provide standard CRUD operations for all entity types.
- **CourseRepository.cs, StudentRepository.cs, etc.:** Specialized repository classes that handle entity-specific data queries.
- **Migrations/:** Contains the database schema evolution history, including the setup for Identity and the various LMS tables.

Models & ViewModels

- **Models/:** Contains domain entities like `Course.cs`, `Lesson.cs`, `Student.cs`, and `Enrollment.cs` which represent the database structure.
- **ViewModels/:** Data Transfer Objects (DTOs) like `StudentCreateViewModel.cs` and `LoginViewModel.cs` used to pass validated data between the views and controllers.
- **ApplicationUser.cs:** Extends the default Identity user to include custom fields like `FirstName`, `LastName`, and `StudentNumber`.

Views & UI

- **Views/Course/, Views/Student/, etc.:** Razor templates for the user interface, supporting CRUD operations and dashboard displays.
- **_Layout.cshtml:** The main master page defining the site structure, navigation bar, and footer.
- **site.css:** Custom styling that defines the "MiniLMS" visual identity, including a custom color palette and responsive grid layouts.
- **site.js:** A placeholder for client-side logic and interactivity.

Configuration & Helpers

- **MappingProfile.cs:** Configures AutoMapper to automatically transform domain models into ViewModels.
- **ExceptionHandlerMiddleware.cs:** Intercepts application errors to provide a consistent error response or logging.
- **launchSettings.json:** Configures the local development environment, including ports and SSL settings.

Repository Structure

```
├─ MiniLms/
│   ├─ Controllers/
│   │   ├─ AccountController.cs
│   │   ├─ CourseController.cs
│   │   ├─ EnrollmentController.cs
│   │   ├─ HomeController.cs
│   │   ├─ LessonController.cs
│   │   ├─ StudentController.cs
│   │   └─ StudentCoursesController.cs
│   ├─ Data/
│   │   └─ ApplicationDbContext.cs
│   ├─ Interfaces/
│   │   ├─ IAiService.cs
│   │   ├─ ICourseDocumentRepository.cs
│   │   ├─ ICourseDocumentService.cs
│   │   ├─ ICourseRepository.cs
│   │   ├─ ICourseService.cs
│   │   ├─ IEnrollmentRepository.cs
│   │   ├─ IEnrollmentService.cs
│   │   ├─ IGenericRepository.cs
│   │   ├─ ILessonContentRepository.cs
│   │   ├─ ILessonContentService.cs
│   │   ├─ ILessonRepository.cs
│   │   ├─ ILessonService.cs
│   │   ├─ IStudentRepository.cs
│   │   ├─ IStudentService.cs
│   │   └─ IVectorDbService.cs
│   ├─ Mappings/
│   │   └─ MappingProfile.cs
│   ├─ Middlewares/
│   │   └─ ExceptionHandlingMiddleware.cs
│   └─ Migrations/
│       ├─ 20260629080034_InitialCreate.cs
│       ├─ 20260629080034_InitialCreate.Designer.cs
│       ├─ 20260629080714_AddEnrollmentTable.cs
│       ├─ 20260629080714_AddEnrollmentTable.Designer.cs
│       ├─ 20260629123118_UpdateCourseColumns.cs
│       ├─ 20260629123118_UpdateCourseColumns.Designer.cs
│       ├─ 20260629124038_FixEnrollmentPrimaryKey.cs
│       ├─ 20260629124038_FixEnrollmentPrimaryKey.Designer.cs
│       ├─ 20260629125338_AddCourseDocumentsTable.cs
│       ├─ 20260629125338_AddCourseDocumentsTable.Designer.cs
│       └─ 20260629133546_AddCourseDocumentsPhysicalTable.cs
```

```
| | └─ 20260629133546_AddCourseDocumentsPhysicalTable.Designer.cs
| | └─ 202606301114801_AddLesson.cs
| | └─ 202606301114801_AddLesson.Designer.cs
| | └─ 20260702111805_SetTableToLessonContents.cs
| | └─ 20260702111805_SetTableToLessonContents.Designer.cs
| | └─ 20260706063603_AddIdentity.cs
| | └─ 20260706063603_AddIdentity.Designer.cs
| | └─ 20260707110700_AddUniqueEnrollmentPerStudentCourse.cs
| | └─ ApplicationDbContextModelSnapshot.cs
├─ Models/
|   │ └─ Enums/
|   │   │ └─ ContentType.cs
|   │   │ └─ UserPolicies.cs
|   │   └─ UserRoles.cs
|   │ └─ ApplicationUser.cs
|   │ └─ Course.cs
|   │ └─ CourseDocument.cs
|   │ └─ Enrollment.cs
|   │ └─ ErrorViewModel.cs
|   │ └─ Lesson.cs
|   │ └─ LessonContent.cs
|   │ └─ Student.cs
├─ Properties/
|   └─ launchSettings.json
├─ Repositories/
|   │ └─ CourseDocumentRepository.cs
|   │ └─ CourseRepository.cs
|   │ └─ EnrollmentRepository.cs
|   │ └─ GenericRepository.cs
|   │ └─ LessonContentRepository.cs
|   │ └─ LessonRepository.cs
|   │ └─ StudentRepository.cs
├─ Services/
|   │ └─ AiService.cs
|   │ └─ CourseDocumentService.cs
|   │ └─ CourseService.cs
|   │ └─ EnrollmentService.cs
|   │ └─ LessonContentService.cs
|   │ └─ LessonService.cs
|   │ └─ StudentService.cs
|   │ └─ VectorDbService.cs
|   │ └─ VectorSyncService.cs
├─ ViewModels/
|   └─ LessonContentViewModel.cs
```

```

├── LessonViewModel.cs
├── LoginViewModel.cs
├── RegisterViewModel.cs
├── StudentCourseListViewModel.cs
├── StudentCreateViewModels.cs
├── StudentEditViewModel.cs
├── Views/
│   ├── Account/
│   │   ├── AccessDenied.cshtml
│   │   ├── Login.cshtml
│   │   └── Register.cshtml
│   ├── Course/
│   │   ├── Create.cshtml
│   │   ├── Create.cshtml.cs
│   │   ├── Delete.cshtml
│   │   ├── Delete.cshtml.cs
│   │   ├── Details.cshtml
│   │   ├── Details.cshtml.cs
│   │   ├── Edit.cshtml
│   │   ├── Edit.cshtml.cs
│   │   ├── Index.cshtml
│   │   └── Index.cshtml.cs
│   ├── Enrollment/
│   │   ├── Create.cshtml
│   │   ├── Create.cshtml.cs
│   │   ├── Delete.cshtml
│   │   ├── Delete.cshtml.cs
│   │   ├── Index.cshtml
│   │   └── Index.cshtml.cs
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   ├── _Layout.cshtml.css
│   │   ├── _ValidationScriptsPartial.cshtml
│   │   └── Error.cshtml
│   └── Student/
│       ├── Create.cshtml
│       ├── Create.cshtml.cs
│       ├── Delete.cshtml
│       ├── Delete.cshtml.cs
│       ├── Details.cshtml
│       └── Details.cshtml.cs

```

```
├── Edit.cshtml
├── Edit.cshtml.cs
├── Index.cshtml
├── Index.cshtml.cs
├── StudentCourses/
│   └── Index.cshtml
├── _ViewImports.cshtml
├── _ViewStart.cshtml
├── wwwroot/
│   ├── css/
│   │   └── site.css
│   ├── js/
│   │   └── site.js
│   ├── lib/
│   │   ├── bootstrap/
│   │   │   └── LICENSE
│   │   ├── jquery/
│   │   │   └── LICENSE.txt
│   │   ├── jquery-validation/
│   │   │   └── LICENSE.md
│   │   └── jquery-validation-unobtrusive/
│   │       └── LICENSE.txt
│   └── uploads/
│       ├── 3f8bea48-4069-4c45-bd50-6fc3e81d56af_ASP.NET ile Web Programlama Eğit
│       ├── 5d493e60-e66f-4bd2-b913-6a71bf8df3c8_SE3004 Software Testing 2025-202
│       └── d719ab98-264f-43e0-b101-c28907cdc667_Lecture 5 - Names, Scope, Bindir
├── appsettings.Development.json
├── appsettings.json
├── MiniLms.csproj
├── Program.cs
├── .gitattributes
├── .gitignore
├── MiniLms.slnx
└── README.md
```

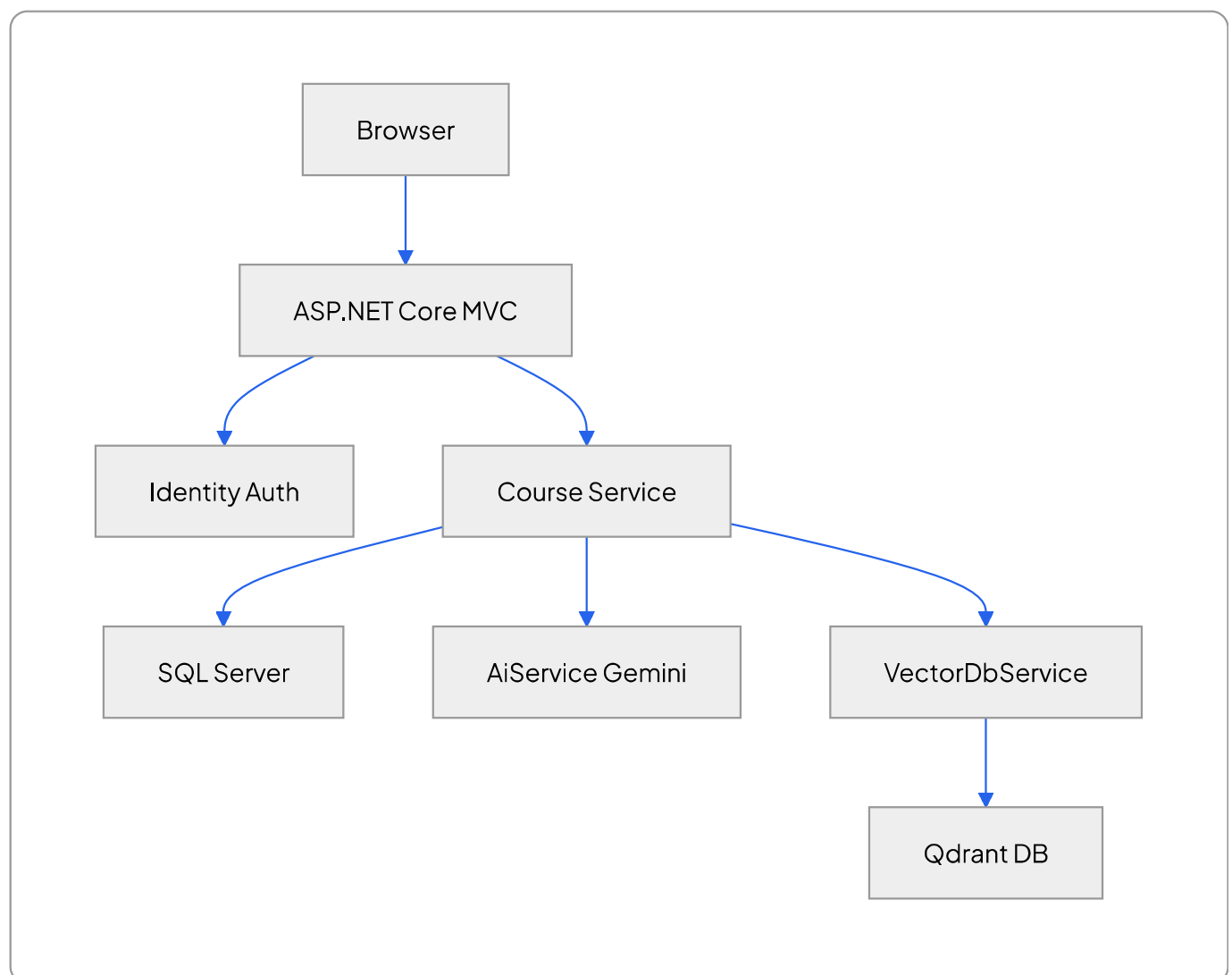
This repository follows a clean, N-tier architecture for a Learning Management System (LMS) built with ASP.NET Core MVC. It organizes core logic into **Models**, **Repositories**, and **Services** to handle data persistence and business rules, while the **Controllers** and **Views** manage the web interface for student enrollments and course management. Notably, the project includes advanced capabilities for AI and vector database integration, alongside standard features like Identity management and file upload handling.

Architecture

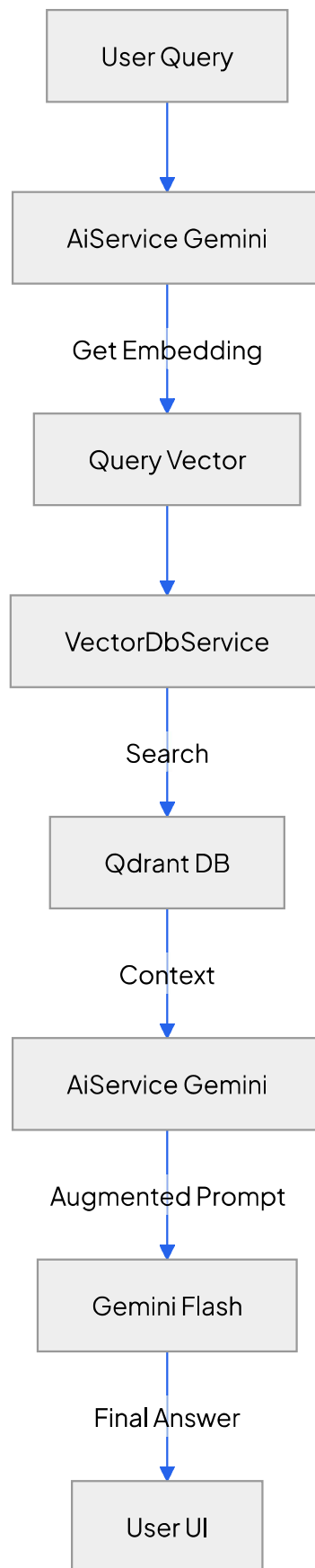
Components

- **ASP.NET Core MVC:** The primary web framework managing the presentation layer, routing, and user identity.
- **SQL Server & EF Core:** Handles persistent storage for structured data like users, courses, and student enrollments.
- **Google Gemini AI:** Provides generative capabilities for content summarization, quiz generation, and text embeddings.
- **Qdrant Vector DB:** A specialized database for storing and querying high-dimensional vectors to enable semantic search.

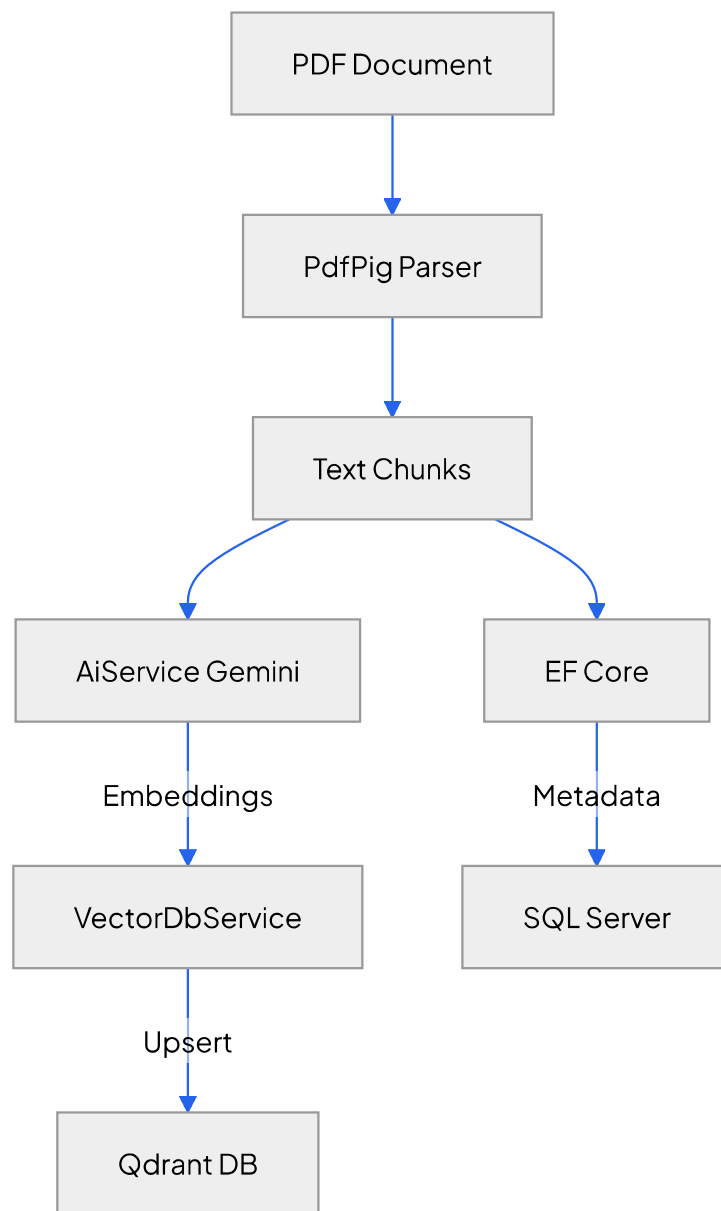
High-Level Architecture



RAG & Semantic Search Pipeline



Content Ingestion Pipeline



Code Quality

This code quality review analyzes the provided architecture and codebase for the **MiniLMS AI** project.

Issues Found

- **Security: Hardcoded Connection String in** `appsettings.json`
 - *File:* `MiniLms/appsettings.json`
 - The connection string contains `Trusted_Connection=True` and `TrustServerCertificate=True`. While acceptable for local development, having these in the main configuration file risks accidental deployment to production.
- **Missing Configuration for Secrets**
 - *File:* `MiniLms/appsettings.json`
 - The architecture relies on **Google Gemini AI** and **Qdrant**, yet there are no placeholders or keys for API authentication (e.g., `ApiKey`). These should never be in `appsettings.json`, but their absence in the schema suggests they might be hardcoded in the service implementations.
- **Localization Inconsistency**
 - *File:* `MiniLms/ViewModels/StudentCreateViewModels.cs`
 - The validation error messages are in Turkish (e.g., `"Ad zorunludur."`), while the property names and the rest of the project documentation are in English. This leads to a fragmented developer and user experience.
- **Potential Model Versioning Issue**
 - *File:* `MiniLms/appsettings.json`
 - The `TextModel` is set to `gemini-2.5-flash`. While you should not flag models as "non-existent," using a specific version string in a shared config without a fallback mechanism can lead to breaking changes if the provider deprecates that specific version.
- **Incomplete Data Annotations**
 - *File:* `MiniLms/ViewModels/StudentCreateViewModels.cs`
 - The `StudentNumber` field is required but lacks a `StringLength` or `RegularExpression` attribute. In an LMS, student IDs usually follow a specific format; lacking validation here could lead to "dirty data" in the SQL database.

- **Architecture/Code Sync Gap**

- *File:* `README.md` vs Project Structure.
- The README claims a "Clean Architecture" (Layers: Presentation, Business, Data Access), but the file structure shown is a standard ASP.NET Core MVC monolithic structure. This is a documentation-to-code mismatch.

Improvement Suggestions

1. **Move Secrets to Environment Variables (High Impact):** Ensure the Gemini API Key and Qdrant credentials are not stored in `appsettings.json`. Use the **Secret Manager** (`dotnet user-secrets`) for development and Environment Variables/Azure Key Vault for production.
2. **Standardize Localization (Medium Impact):** Decide on a single language for code-level strings. If the app is multi-language, use `.resx` files for `ErrorMessage` values in ViewModels instead of hardcoded strings.
3. **Enhance Validation Logic (Medium Impact):** Add `[StringLength]` and format validation to the `StudentNumber` in `StudentCreateViewModel` to ensure data integrity before it reaches the Service layer.
4. **Refine Configuration Schema (Low Impact):** Group AI settings more logically. For example, include a `BaseUrl` for Qdrant and a `RetryCount` for AI API calls to make the system more resilient to network flakiness.
5. **Update Documentation (Low Impact):** Align the README "Presentation/Business/Data" tree with the actual folder structure to avoid confusing new contributors.

Overall Quality Score

Score: 7/10

Justification: The project demonstrates a very strong architectural foundation, particularly the integration of RAG (Retrieval-Augmented Generation) and vector databases within a traditional .NET ecosystem. The use of ViewModels and the Repository pattern (as mentioned in docs) follows industry best practices. The score is held back by minor inconsistencies in localization and the risk of sensitive configuration management.

Feature Suggestions

Based on the current architecture of MiniLMS AI, here are the recommended next steps to improve the product's utility and technical robustness.

Quick Wins (Low Effort, High Impact)

1. **Automated Quiz Generation** Use Gemini to generate multiple-choice questions based on the text chunks stored in SQL/Qdrant for a specific course. This provides immediate value to educators by automating the creation of assessment materials.
2. **Citation Links in RAG Responses** Modify the RAG pipeline to return the specific source metadata (e.g., PDF page number or chunk ID) alongside the AI answer. This builds user trust by allowing students to verify AI-generated answers against the original course material.
3. **Course "Flashcard" Generator** Create a feature that extracts key terms and definitions from the vector database to generate study sets. This leverages existing data to provide a new study tool for students with minimal backend changes.

Medium Effort (Moderate Complexity)

4. **Asynchronous Document Processing** Move the PDF parsing and embedding logic into a background worker (using Hangfire or Coravel). This prevents the UI from freezing during large file uploads and improves the overall perceived performance of the application.
5. **Conversation History for AI Chat** Store chat sessions in SQL Server to provide context-aware follow-up questions. This transforms the current single-turn Q&A into a true "tutor" experience where the AI remembers previous parts of the conversation.
6. **Semantic Search for Course Discovery** Extend the Qdrant implementation to allow students to search for entire courses based on concepts rather than just keywords. This helps students find relevant learning materials even if they don't know the exact course titles.
7. **Support for Markdown and Office Files** Expand the ingestion pipeline beyond PdfPig to include libraries like Markdig or OpenXML. This allows educators to upload a wider variety of existing teaching materials without manual conversion.

Larger Initiatives (High Impact, Strategic)

8. **Student Progress Analytics via AI** Analyze student quiz results and chat queries to identify "knowledge gaps" and report them to the instructor. This provides educators with actionable insights into which topics the class is struggling to understand.

9. **Multi-Modal Support (Images/Diagrams)** Leverage Gemini's vision capabilities to process diagrams and charts within the uploaded PDFs. This ensures that visual learners and technical subjects (like engineering or biology) are better served by the RAG system.
10. **Hybrid Search Implementation** Combine Qdrant's vector search with SQL Server's Full-Text Search using a re-ranking algorithm. This improves accuracy for queries involving specific technical terms or acronyms that embeddings might occasionally generalize too much.